

VISVESVARAYA TECHNOLOGICAL UNIVERSITY
“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT
on
OPERATING SYSTEMS

Submitted by

NITYA R SWADI (1BM24CS361)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
Feb-2026 to June-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “OPERATING SYSTEMS – 23CS4PCOPS” carried out by Nitya R Swadi(1BM24CS361), who is a Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026.

The Lab report has been approved as it satisfies the academic requirements in respect of a OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Faculty Incharge Name
Dr. Kavitha Sooda
Assistant Professor
Professor and Head
Department of CSE
Department of CSE
BMSCE, Bengaluru
Bengaluru

BMSCE,

Index Sheet

Sl. No.	Experiment Title	Page No.
1.	Write a C program to simulate the following non-pre-emptive CPU scheduling algorithm to find turnaround time and waiting time. →FCFS → SJF (pre-emptive & Non-preemptive)	4-14
2.	Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. → Priority (pre-emptive & Non-pre-emptive) →Round Robin (Experiment with different quantum sizes for RR algorithm)	15-25
3.	Write a C program to simulate a multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.	26-30
4.	Write a C program to simulate Real-Time CPU Scheduling algorithms: a) Rate- Monotonic b) Earliest-deadline First c) Proportional scheduling	31-43
5.	Write a C program to simulate producer-consumer problem using semaphores	44-48
6.	Write a C program to simulate the concept of Dining Philosophers problem.	49-51
7.	Write a C program to simulate Bankers algorithm for the purpose of deadlock avoidance.	52-62

Course Outcomes

C01	Apply the different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating Systems.
C04	Conduct practical experiments to implement the functionalities of Operating systems.

Lab Program 1

FCFS:

```
//first come first serve
#include <stdio.h>

int main() {
    int n, i, j;

    printf("Enter total number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], ct[n], tat[n], wt[n], pid[n];
    float avg_tat = 0, avg_wt = 0;

    // Input Arrival Time and Burst Time
    for(i = 0; i < n; i++) {
        pid[i] = i + 1;
        printf("\nProcess %d\n", pid[i]);
        printf("Arrival Time: ");
        scanf("%d", &at[i]);
        printf("Burst Time: ");
        scanf("%d", &bt[i]);
    }

    // Sort processes by Arrival Time
    for(i = 0; i < n - 1; i++)
    {
        for(j = i + 1; j < n; j++) {
            if(at[i] > at[j]) {
                int temp;
                temp = at[i];
                at[i] = at[j];
                at[j] = temp;
            }
        }
    }

    // Calculate waiting time and turn around time
    ct[0] = bt[0];
    for(i = 1; i < n; i++)
        ct[i] = ct[i-1] + bt[i];

    for(i = 0; i < n; i++)
        tat[i] = ct[i];

    for(i = 0; i < n; i++)
        wt[i] = tat[i] - bt[i];

    avg_tat = (tat[0] + tat[1] + ... + tat[n-1]) / n;
    avg_wt = (wt[0] + wt[1] + ... + wt[n-1]) / n;

    printf("\nAverage Turn Around Time: %.2f", avg_tat);
    printf("\nAverage Waiting Time: %.2f", avg_wt);
}
```

```

        at[j] = temp;
        temp = bt[i];
        bt[i] = bt[j];
        bt[j] = temp;
        temp = pid[i];
        pid[i] = pid[j];
        pid[j] = temp;
    }
}

int current_time = 0;
// Calculate Completion Time, Turnaround Time, Waiting Time
for(i = 0; i < n; i++) {
    if(current_time < at[i])
        current_time = at[i];
    ct[i] = current_time + bt[i];
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
    current_time = ct[i];
    avg_tat += tat[i];
    avg_wt += wt[i];
}

avg_tat /= n;
avg_wt /= n;
// Output
printf("\nProcess\tArrival Time\tBurst Time\tCompletion Time\tTurnaround Time\tWaiting Time\n");
for(i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t\t%d\t\t%d\t\t%d\n", pid[i], at[i], bt[i], ct[i], tat[i], wt[i]);
}

```

```

}
printf("\nAverage Turnaround Time: %.2f", avg_tat);
printf("\nAverage Waiting Time: %.2f\n", avg_wt);
return 0;
}

```

OUTPUT:

```

C:\Users\Nitya\OneDrive\Desktop X + v
IBM24CS361 Nitya R Swadi

Enter total number of processes: 4

Process 1
Arrival Time: 2
Burst Time: 9

Process 2
Arrival Time: 0
Burst Time: 4

Process 3
Arrival Time: 4
Burst Time: 3

Process 4
Arrival Time: 1
Burst Time: 2

Process Arrival Time Burst Time Completion Time Turnaround Time Waiting Time
P2 0 4 4 4 0
P4 1 2 6 5 3
P1 2 9 15 13 4
P3 4 3 18 14 11

Average Turnaround Time: 9.00
Average Waiting Time: 4.50

Process returned 0 (0x0) execution time : 24.125 s
Press any key to continue.

```

SJF Non preemptive:

```
#include <stdio.h>

int main() {
    int n, i, j;

    printf("Enter number of processes: ");
    if (scanf("%d", &n) != 1) return 1;

    int at[n], bt[n], ct[n], tat[n], wt[n], pid[n];
    int finished[n];

    float avg_tat = 0, avg_wt = 0;

    // Input arrival times
    printf("Enter arrival times:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &at[i]);
        pid[i] = i + 1;
        finished[i] = 0;
    }

    // Input burst times
    printf("Enter burst times:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &bt[i]);
    }
```

```

int completed = 0;
int current_time = 0;

while(completed < n) {
    int min_bt = 9999;
    int index = -1;

    // Find process with minimum burst time in the ready queue
    for(i = 0; i < n; i++) {
        if(at[i] <= current_time && finished[i] == 0) {
            if(bt[i] < min_bt) {
                min_bt = bt[i];
                index = i;
            }
        }
    }

    // If no process is ready, advance time (CPU Idle)
    if(index == -1) {
        current_time++;
    }
    else {
        // Calculate metrics for the selected process
        ct[index] = current_time + bt[index];
        tat[index] = ct[index] - at[index];
        wt[index] = tat[index] - bt[index];

        current_time = ct[index];
    }
}

```



```

        finished[index] = 1;
        completed++;

        avg_tat += tat[index];
        avg_wt += wt[index];
    }
}

avg_tat /= n;
avg_wt /= n;

// Output Results Table
printf("\nSJF Scheduling (Non-Preemptive):\n");
printf("-----\n");
printf("PID\tAT\tBT\tCT\tTAT\tWT\n");
printf("-----\n");
for(i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
           pid[i], at[i], bt[i], ct[i], tat[index], wt[i]);
}
printf("-----\n");

printf("\nAverage Turnaround Time: %.2f ms\n", avg_tat);
printf("Average Waiting Time: %.2f ms\n", avg_wt);

return 0;
}

```

OUTPUT:

```
C:\Users\Nitya\OneDrive\Des X + v
1BM24CS361 Nitya R Swadi

Enter number of processes: 3
Enter arrival times:
0 2 1
Enter burst times:
6 5 3

SJF Scheduling (Non-Preemptive):
-----
PID      AT      BT      CT      TAT      WT
-----
P1        0        6        6        6        0
P2        2        5       14       12        7
P3        1        3        9        8        5
-----

Average Turnaround Time: 8.67 ms
Average Waiting Time: 4.00 ms

Process returned 0 (0x0)   execution time : 18.239 s
Press any key to continue.
|
```

SJF Preemptive (SRTF):

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, i;
```

```
    printf("Enter number of processes: ");
```

```
    if (scanf("%d", &n) != 1) return 1;
```

```
    int at[n], bt[n], rt[n], ct[n], tat[n], wt[n], pid[n];
```

```
    // Input loop for process details
```

```
    for(i = 0; i < n; i++) {
```

```
        pid[i] = i + 1;
```

```
        printf("\nProcess P%d\n", pid[i]);
```

```
        printf("Arrival Time: ");
```

```
        scanf("%d", &at[i]);
```

```
        printf("Burst Time: ");
```

```
        scanf("%d", &bt[i]);
```

```
        rt[i] = bt[i]; // Remaining time initially equals burst time
```

```
    }
```

```
    int completed = 0;
```

```
    int current_time = 0;
```

```
    int min_index;
```

```
    int min_rt;
```

```
    float avg_tat = 0, avg_wt = 0;
```

```
// Core SRTF Scheduling loop
```

```
while(completed < n) {
```

```
    min_rt = 9999;
```

```
    min_index = -1;
```

```
// Find process with the smallest non-zero remaining time among ready processes
```

```
for(i = 0; i < n; i++) {
```

```
    if(at[i] <= current_time && rt[i] > 0) {
```

```
        if(rt[i] < min_rt) {
```

```
            min_rt = rt[i];
```

```
            index = i; // Note: fixed variable name from your snippet logic
```

```
            min_index = i;
```

```
        }
```

```
    }
```

```
}
```

```
// If no process is ready, advance time (CPU Idle)
```

```
if(min_index == -1) {
```

```
    current_time++;
```

```
}
```

```
else {
```

```
    // Execute the process with the shortest remaining time for 1 time unit
```

```
    rt[min_index]--;
```

```
    current_time++;
```

```
// Check if the process has finished executing
```

```
if(rt[min_index] == 0) {
```

```
    completed++;
```

```

        ct[min_index] = current_time;
        tat[min_index] = ct[min_index] - at[min_index];
        wt[min_index] = tat[min_index] - bt[min_index];

        avg_tat += tat[min_index];
        avg_wt += wt[min_index];
    }
}
}

```

```

// Calculate averages

```

```

avg_tat /= n;

```

```

avg_wt /= n;

```

```

// Output Results Table

```

```

printf("\nSJF Preemptive (SRTF) Scheduling:\n");
printf("-----\n");
printf("PID\tAT\tBT\tCT\tTAT\tWT\n");
printf("-----\n");
for(i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        pid[i], at[i], bt[i], ct[i], tat[i], wt[i]);
}
printf("-----\n");

```

```

printf("\nAverage Turnaround Time: %.2f ms\n", avg_tat);

```

```

printf("Average Waiting Time: %.2f ms\n", avg_wt);

```

```

return 0;

```

}

OUTPUT:

```
C:\Users\Nitya\OneDrive\Des  X + v
1BM24CS361 Nitya R Swadi
Enter number of processes: 3

Process P1
Arrival Time: 0
Burst Time: 5

Process P2
Arrival Time: 1
Burst Time: 2

Process P3
Arrival Time: 2
Burst Time: 1

SJF Preemptive (SRTF) Scheduling:
-----
PID      AT      BT      CT      TAT      WT
-----
P1        0        5        8        8        3
P2        1        2        3        2        0
P3        2        1        4        2        1
-----

Average Turnaround Time: 4.00 ms
Average Waiting Time: 1.33 ms

Process returned 0 (0x0)   execution time : 13.953 s
Press any key to continue.
```

Lab Program 2

Priority Scheduling Non-Preemptive:

```
#include <stdio.h>
#include <stdbool.h>

int main() {
    int n;
    printf("Nitya R Swadi, 1BM24CS361\n");

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int arrival_time[n], burst_time[n], priority[n];
    int completion_time[n], turnaround_time[n], waiting_time[n];
    bool completed[n];

    for (int i = 0; i < n; i++) {
        printf("\nProcess P%d\n", i + 1);

        printf("Enter Arrival Time:
"); scanf("%d",
&arrival_time[i]);

        printf("Enter Burst Time:
"); scanf("%d",
&burst_time[i]);
```

```

printf("Enter Priority:
"); scanf("%d",
&priority[i]);

completed[i] = false;

}

int current_time = 0;
int completed_count = 0;

while (completed_count < n) {

    int highest_priority =
    9999; int selected_process
    = -1;

    for (int i = 0; i < n; i++) {
        if (arrival_time[i] <= current_time && !completed[i]) {

            if (priority[i] < highest_priority) {
                highest_priority = priority[i];
                selected_process = i;
            }
        }
    }

    if (selected_process == -1)
        { current_time++;
        }
    else {

```



```

int start_time = current_time;

completion_time[selected_process] =
    start_time + burst_time[selected_process];

turnaround_time[selected_process] =
    completion_time[selected_process]
    - arrival_time[selected_process];

waiting_time[selected_process] =
    turnaround_time[selected_process]
    - burst_time[selected_process];

current_time = completion_time[selected_process];

completed[selected_process] = true;
completed_count++;
}
}

float total_wt = 0, total_tat = 0;

printf("\nProcess\tAT\tBT\tPriority\tWT\tTAT\n");

for (int i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t\t%d\t%d\n",
        i + 1,
        arrival_time[i],
        burst_time[i],
        priority[i],

```

```

        waiting_time[i],
        turnaround_time[i]);

    total_wt += waiting_time[i];
    total_tat += turnaround_time[i];
}

printf("\nAverage Waiting Time = %.2f", total_wt / n);
printf("\nAverage Turnaround Time = %.2f\n", total_tat / n);

```

OUTPUT:

```

C:\Users\Admin\Desktop\Nitya R Swadi, 1BM24CS361
Nitya R Swadi, 1BM24CS361
Enter number of processes: 3

Process P1
Enter Arrival Time: 0
Enter Burst Time: 2
Enter Priority: 4

Process P2
Enter Arrival Time: 1
Enter Burst Time: 5
Enter Priority: 1

Process P3
Enter Arrival Time: 1
Enter Burst Time: 3
Enter Priority: 2

Process AT    BT    Priority    WT    TAT
P1    0    2    4    0    2
P2    1    5    1    1    6
P3    1    3    2    6    9

Average Waiting Time = 2.33
Average Turnaround Time = 5.67

Process returned 0 (0x0)   execution time : 31.275 s
Press any key to continue.

```

Priority Scheduling Preemptive:

```
#include <stdio.h>

int main() {
    int n;

    printf("Nitya R Swadi, 1BM24CS361");

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int arrival_time[n], burst_time[n], priority[n];
    int remaining_time[n];
    int completion_time[n], turnaround_time[n], waiting_time[n];

    for(int i = 0; i < n; i++) {
        printf("\nProcess P%d\n", i + 1);

        printf("Enter Arrival Time:
"); scanf("%d",
&arrival_time[i]);

        printf("Enter Burst Time:
"); scanf("%d",
&burst_time[i]);

        printf("Enter Priority:
"); scanf("%d",
&priority[i]);
```

```

    remaining_time[i] = burst_time[i];
}

int current_time = 0;
int completed = 0;

while(completed < n) {
    int highest_priority = 9999;
    int selected_process = -1;

    for(int i = 0; i < n; i++) {

        if(arrival_time[i] <= current_time && remaining_time[i] > 0) {

            if(priority[i] < highest_priority) {
                highest_priority = priority[i];
                selected_process = i;
            }
        }
    }

    if(selected_process == -1)
        { current_time++;
        }
    else {

        remaining_time[selected_process]--;

        current_time++;
    }
}

```

```

if(remaining_time[selected_process] == 0) {

    completion_time[selected_process] = current_time;
    turnaround_time[selected_process] =
    completion_time[selected_process] - arrival_time[selected_process];

    waiting_time[selected_process] =
    turnaround_time[selected_process] - burst_time[selected_process];
    completed++;
}
}
}

```

```

float total_wt = 0, total_tat = 0;

```

```

printf("\nProcess\tAT\tBT\tPriority\tWT\tTAT\n");

```

```

for(int i = 0; i < n; i++) {

```

```

    printf("P%d\t%d\t%d\t%d\t%d\t%d\n"

```

```

        , i + 1,

```

```

        arrival_time[i],

```

```

        burst_time[i],

```

```

        priority[i],

```

```

        waiting_time[i],

```

```

        turnaround_time[i]);

```

```

total_wt += waiting_time[i];

```

```

total_tat += turnaround_time[i];

```

```

    }

    printf("\nAverage Waiting Time = %.2f", total_wt / n);
    printf("\nAverage Turnaround Time = %.2f\n", total_tat / n);

    return 0;
}

```

OUTPUT:

```

C:\Users\Admin\Desktop\Nitya X + v
Nitya R Swadi, 1BM24CS361
Enter number of processes: 3

Process P1
Enter Arrival Time: 0
Enter Burst Time: 3
Enter Priority: 5

Process P2
Enter Arrival Time: 1
Enter Burst Time: 5
Enter Priority: 1

Process P3
Enter Arrival Time: 2
Enter Burst Time: 3
Enter Priority: 2

Process AT      BT      Priority      WT      TAT
P1      0      3      5      8      11
P2      1      5      1      0      5
P3      2      3      2      4      7

Average Waiting Time = 4.00
Average Turnaround Time = 7.67

Process returned 0 (0x0)   execution time : 39.658 s
Press any key to continue.

```

Round Robin Scheduling:

```
#include <stdio.h>
```

```
int main() {
```

```

int n, tq;

printf("Nitya R Swadi, 1BM24CS361\n");

printf("Enter number of processes: ");
scanf("%d", &n);

printf("Enter Time Quantum: ");
scanf("%d", &tq);

int at[n], bt[n], rt[n];
int wt[n], tat[n];
int completed = 0, current_time = 0;

for(int i = 0; i < n; i++) {
    printf("\nProcess P%d\n", i + 1);

    printf("Enter Arrival Time: ");
    scanf("%d", &at[i]);

    printf("Enter Burst Time: ");
    scanf("%d", &bt[i]);

    rt[i] = bt[i]; // remaining time
}

while(completed < n) {

    int done = 1;

    for(int i = 0; i < n; i++) {

```

```

if(at[i] <= current_time && rt[i] > 0) {

    done = 0;

    if(rt[i] > tq) {
        current_time += tq; rt[i] -= tq;

    } else {

        current_time += rt[i];

        tat[i] = current_time -
        at[i]; wt[i] = tat[i] - bt[i];

        rt[i] = 0;
        completed++;
    }
}

if(done)
    current_time++;
}

float total_wt = 0, total_tat = 0;

printf("\nProcess\tAT\tBT\tWT\tTAT\n");

for(int i = 0; i < n; i++) {

```



```

printf("P%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], wt[i],
tat[i]);

total_wt += wt[i];
total_tat += tat[i];
}

printf("\nAverage Waiting Time = %.2f", total_wt / n);
printf("\nAverage Turnaround Time = %.2f\n", total_tat / n);

return 0;
}

```

OUTPUT:

```

C:\Users\Admin\Desktop\Nitya > Nitya R Swadi, 1BM24CS361
Enter number of processes: 3
Enter Time Quantum: 2

Process P1
Enter Arrival Time: 0
Enter Burst Time: 4

Process P2
Enter Arrival Time: 0
Enter Burst Time: 3

Process P3
Enter Arrival Time: 1
Enter Burst Time: 2

Process AT      BT      WT      TAT
P1      0      4      4      8
P2      0      3      6      9
P3      1      2      3      5

Average Waiting Time = 4.33
Average Turnaround Time = 7.33

Process returned 0 (0x0)   execution time : 14.735 s
Press any key to continue.

```

Lab Program 3

Multilevel Queue Scheduling:

```
#include <stdio.h>
#include <string.h>

#define MAX 100

typedef struct {
    int pid;
    int arrival;
    int burst;
    int completion;
    int turnaround;
    int waiting;
    char type[10]; // "System" or "User"
} Process;

int main() {
    Process sysQ[MAX], userQ[MAX];
    int n, i, sCount = 0, uCount = 0;

    printf("Enter number of processes: ");
    if (scanf("%d", &n) != 1) return 1;

    Process temp;

    // --- Input Process Details ---
```

```

for(i = 0; i < n; i++) {
    printf("\nProcess %d\n", i + 1);
    printf("Enter Process ID: ");
    scanf("%d", &temp.pid);

    printf("Enter Arrival Time: ");
    scanf("%d", &temp.arrival);

    printf("Enter Burst Time: ");
    scanf("%d", &temp.burst);

    printf("Enter Type (System/User): ");
    scanf("%s", temp.type);

    // Split processes into their respective priority queues
    if(strcmp(temp.type, "System") == 0) {
        sysQ[sCount++] = temp;
    } else {
        userQ[uCount++] = temp;
    }
}

int current_time = 0;

// --- Execute System Queue (Higher Priority) ---
for(i = 0; i < sCount; i++) {
    if(current_time < sysQ[i].arrival) {
        current_time = sysQ[i].arrival;
    }
}

```

```

    current_time += sysQ[i].burst;
    sysQ[i].completion = current_time;

    sysQ[i].turnaround = sysQ[i].completion - sysQ[i].arrival;
    sysQ[i].waiting = sysQ[i].turnaround - sysQ[i].burst;
}

// --- Execute User Queue (Lower Priority) ---
for(i = 0; i < uCount; i++) {
    if(current_time < userQ[i].arrival) {
        current_time = userQ[i].arrival;
    }

    current_time += userQ[i].burst;
    userQ[i].completion = current_time;

    userQ[i].turnaround = userQ[i].completion - userQ[i].arrival;
    userQ[i].waiting = userQ[i].turnaround - userQ[i].burst;
}

// --- Output Results ---
float totalWT = 0, totalTAT = 0;

printf("\n=====");
printf("\n      SYSTEM PROCESSES      ");
printf("\n=====\\n");
printf("PID\\tAT\\tBT\\tCT\\tTAT\\tWT\\n");
printf("-----\\n");

```

```

for(i = 0; i < sCount; i++) {
    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        sysQ[i].pid,
        sysQ[i].arrival,
        sysQ[i].burst,
        sysQ[i].completion,
        sysQ[i].turnaround,
        sysQ[i].waiting);

    totalWT += sysQ[i].waiting;
    totalTAT += sysQ[i].turnaround;
}

printf("\n=====");
printf("\n      USER PROCESSES      ");
printf("\n=====\\n");
printf("PID\\tAT\\tBT\\tCT\\tTAT\\tWT\\n");
printf("-----\\n");
for(i = 0; i < uCount; i++) {
    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        userQ[i].pid,
        userQ[i].arrival,
        userQ[i].burst,
        userQ[i].completion,
        userQ[i].turnaround,
        userQ[i].waiting);

    totalWT += userQ[i].waiting;
    totalTAT += userQ[i].turnaround;
}

```

```

    }

    printf("-----\n");

    int totalProcesses = sCount + uCount;

    printf("\nAverage Waiting Time   = %.2f ms", totalWT / totalProcesses);
    printf("\nAverage Turnaround Time = %.2f ms\n", totalTAT / totalProcesses);

    return 0;
}

```

OUTPUT:

```

C:\Users\Nitya\OneDrive\Des
Nitya Swadi, 1BM24CS361
Enter number of processes: 3

Process 1
Enter Process ID: 1
Enter Arrival Time: 0
Enter Burst Time: 3
Enter Type (System/User): System

Process 2
Enter Process ID: 2
Enter Arrival Time: 0
Enter Burst Time: 4
Enter Type (System/User): User

Process 3
Enter Process ID: 3
Enter Arrival Time: 1
Enter Burst Time: 6
Enter Type (System/User): User

--- System Processes ---
PID    AT    BT    CT    TAT    WT
1       0     3     3     3     0

--- User Processes ---
PID    AT    BT    CT    TAT    WT
2       0     4     7     7     3
3       1     6    13    12     6

Average Waiting Time = 3.00
Average Turnaround Time = 7.33

Process returned 0 (0x0)   execution time : 51.008 s
Press any key to continue.

```

Lab Program 4

Rate monotonic scheduling:

```
#include <stdio.h>
#include <math.h>
#include <stdlib.h>

#define MAX 100

typedef struct {
    int id;
    int burst;
    int period;
    int remaining_time;
    int is_ready;
} Task;

// Function to find Great Common Divisor (GCD)
int gcd(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

// Function to find Least Common Multiple (LCM)
int find_lcm(int a, int b) {
    return (a * b) / gcd(a, b);
}

int main() {
    int n;

    // Header preservation
    printf("1BM24CS361, Nitya R Swadi\n");
    printf("-----\n");
```

```

printf("Enter the number of processes: ");
if (scanf("%d", &n) != 1 || n <= 0) {
    printf("Invalid number of processes.\n");
    return 1;
}

```

```

Task tasks[n];

```

```

printf("Enter the CPU burst times:\n");
for (int i = 0; i < n; i++) {
    tasks[i].id = i + 1;
    scanf("%d", &tasks[i].burst);
}

```

```

printf("Enter the time periods:\n");
for (int i = 0; i < n; i++) {
    scanf("%d", &tasks[i].period);
    tasks[i].remaining_time = 0;
    tasks[i].is_ready = 0;
}

```

```

// Sort tasks based on Period using Bubble Sort (Shorter period = Higher Priority)
for (int i = 0; i < n - 1; i++) {
    for (int j = 0; j < n - i - 1; j++) {
        if (tasks[j].period > tasks[j + 1].period) {
            Task temp = tasks[j];
            tasks[j] = tasks[j + 1];
            tasks[j + 1] = temp;
        }
    }
}

```

```

// Calculate Hyperperiod (LCM of all periods)
int hyperperiod = tasks[0].period;
for (int i = 1; i < n; i++) {
    hyperperiod = find_lcm(hyperperiod, tasks[i].period);
}
printf("\nHyperperiod (LCM) = %d ms\n\n", hyperperiod);

```

```

// --- Rate Monotone Table Output ---

```



```

printf("Rate Monotone Scheduling Priorities:\n");
printf("-----\n");
printf("PID\tBurst\tPeriod\n");
printf("-----\n");
for (int i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\n", tasks[i].id, tasks[i].burst, tasks[i].period);
}
printf("-----\n\n");

// --- Schedulability Check ---
double utilization = 0.0;
for (int i = 0; i < n; i++) {
    utilization += (double)tasks[i].burst / tasks[i].period;
}
double bound = n * (pow(2.0, 1.0 / n) - 1.0);

printf("CPU Utilization Check:\n");
printf("Utilization: %f | Bound: %f\n", utilization, bound);
printf("Condition (%f <= %f) -> ", utilization, bound);

if (utilization <= bound) {
    printf("TRUE (System is guaranteed to be schedulable)\n");
} else {
    printf("FALSE (System might not be schedulable, but simulating anyway)\n");
}
printf("Scheduling Simulation for %d ms:\n\n", hyperperiod);

// --- Simulation Execution ---
int current_time = 0;
int last_running_task_id = -2;

while (current_time < hyperperiod) {
    // Release tasks arriving at the current time unit
    for (int i = 0; i < n; i++) {
        if (current_time % tasks[i].period == 0) {
            tasks[i].remaining_time = tasks[i].burst;
            tasks[i].is_ready = 1;
        }
    }
}

```

```

// Find the highest priority ready task (first one found due to sorting)
int selected_task_index = -1;
for (int i = 0; i < n; i++) {
    if (tasks[i].is_ready && tasks[i].remaining_time > 0) {
        selected_task_index = i;
        break;
    }
}

// Execute the chosen task
if (selected_task_index != -1) {
    if (last_running_task_id != tasks[selected_task_index].id) {
        printf("%dms onwards: Process P%d running\n", current_time,
tasks[selected_task_index].id);
        last_running_task_id = tasks[selected_task_index].id;
    }

    tasks[selected_task_index].remaining_time--;

    if (tasks[selected_task_index].remaining_time == 0) {
        tasks[selected_task_index].is_ready = 0;
    }
} else {
    // CPU Idle state detection
    if (last_running_task_id != -1) {
        printf("%dms onwards: CPU is idle\n", current_time);
        last_running_task_id = -1;
    }
}

current_time++;
}

return 0;
}

```

OUTPUT:

```
C:\Users\Nitya\Downloads\rm X + v
1BM24CS361, Nitya R Swadi
Enter the number of processes: 2
Enter the CPU burst times:
20 35
Enter the time periods:
50 100
LCM=100

Rate Monotone Scheduling:
PID      Burst   Period
1         20     50
2         35    100

0.750000 <= 0.828427 =>true
Scheduling occurs for 100 ms

0ms onwards: Process 1 running
20ms onwards: Process 2 running
50ms onwards: Process 1 running
70ms onwards: Process 2 running
75ms onwards: CPU is idle

Process returned 0 (0x0)   execution time : 21.226 s
Press any key to continue.
|
```

Earliest Deadline First scheduling:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define INFINITY 999999
```

```
typedef struct {
```

```
    int id;
```

```
    int arrival;
```

```
    int burst;
```

```
    int deadline;
```

```
    int period;
```

```

    int remaining;
    int current_deadline;
} Process;

int main() {
    int n;

    // Header preservation
    printf("1BM24CS361, Nitya R
Swadi\n");

    printf("-----\n");

    printf("Enter the number of processes:
");
    if (scanf("%d", &n) != 1 || n <= 0) {
        printf("Invalid number of
processes.\n");
        return 1;
    }

    Process proc[n];

    // --- Input Process Details ---
    for (int i = 0; i < n; i++) {
        proc[i].id = i + 1;
        printf("\nEnter details for Process
P%d:\n", proc[i].id);
        printf("Arrival, Burst, Deadline,

```

```

Period (Space Separated): ");
    scanf("%d %d %d %d",
&proc[i].arrival, &proc[i].burst,
&proc[i].deadline, &proc[i].period);

    proc[i].remaining = 0;
    proc[i].current_deadline =
proc[i].deadline;
}

// --- Configuration Display ---
printf("\nEDF Input Configuration
Matrix:\n");

printf("-----\n");

printf("PID\tBurst\tDeadline\tPeriod\n");

printf("-----\n");

for (int i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\n",
proc[i].id, proc[i].burst, proc[i].deadline,
proc[i].period);
}

printf("-----\n");

```

```

    int simulation_time = 20;
    printf("Scheduling occurs for %d
ms\n\n", simulation_time);
    printf("Gantt Chart Execution
Timeline:\n");

    printf("-----\n");

    int time = 0;

    // --- Simulation Core Loop ---
    while (time < simulation_time) {

        // Update task instances based on
their periods
        for (int i = 0; i < n; i++) {
            if (time >= proc[i].arrival &&
(time - proc[i].arrival) % proc[i].period
== 0) {
                proc[i].remaining =
proc[i].burst;

                int current_instance = (time -
proc[i].arrival) / proc[i].period;
                proc[i].current_deadline =
proc[i].arrival + (current_instance *
proc[i].period) + proc[i].deadline;
            }
        }
    }

```

```

int min_deadline = INFINITY;
int selected = -1;

// Choose the ready task with the
earliest absolute deadline
for (int i = 0; i < n; i++) {
    if (time >= proc[i].arrival &&
proc[i].remaining > 0) {
        if (proc[i].current_deadline <
min_deadline) {
            min_deadline =
proc[i].current_deadline;
            selected = i;
        }
    }
}

// Process Execution Logging
if (selected == -1) {
    printf("%d ms : CPU is idle.\n",
time);
} else {
    printf("%d ms : Task P%d is
running.\n", time, proc[selected].id);
    proc[selected].remaining =
proc[selected].remaining - 1;
}
time = time + 1;
}

```

```

printf("-----\n");

return 0;
}

```

OUTPUT:

```

18M24CS361, Nitya R Swadi
Enter the number of processes: 3
Enter arrival, burst, deadline, and period for Process 1: 0 3 7 20
Enter arrival, burst, deadline, and period for Process 2: 0 2 4 5
Enter arrival, burst, deadline, and period for Process 3: 0 2 8 10

PID      Burst  Deadline  Period
1         3       7         20
2         2       4          5
3         2       8         10
Scheduling occurs for 20 ms

0ms : Task 2 is running.
1ms : Task 2 is running.
2ms : Task 1 is running.
3ms : Task 1 is running.
4ms : Task 1 is running.
5ms : Task 3 is running.
6ms : Task 3 is running.
7ms : Task 2 is running.
8ms : Task 2 is running.
9ms : CPU is idle.
10ms : Task 2 is running.
11ms : Task 2 is running.
12ms : Task 3 is running.
13ms : Task 3 is running.
14ms : CPU is idle.
15ms : Task 2 is running.
16ms : Task 2 is running.
17ms : CPU is idle.
18ms : CPU is idle.
19ms : CPU is idle.

Process returned 0 (0x0)   execution time : 23.368 s
Press any key to continue.

```


Proportional Share scheduling:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

typedef struct {
    char name[5];
    int tickets;
} Process;

int main() {
    int n, total_tickets = 0;
    float total_T = 0.0;

    // Header preservation
    printf("1BM24CS361, Nitya R Swadi\n");
    printf("-----\n");

    printf("Enter the number of Processes: ");
    if (scanf("%d", &n) != 1 || n <= 0) {
        printf("Invalid number of processes.\n");
        return 1;
    }

    Process p[n];
    srand(time(NULL)); // Seed the random number generator

    // --- Input Process Details ---
    for (int i = 0; i < n; i++) {
        printf("\nProcess %d:\n", i + 1);
        sprintf(p[i].name, "P%d", i + 1);
        printf("Tickets allocated: ");
        scanf("%d", &p[i].tickets);

        total_tickets += p[i].tickets;
        total_T += p[i].tickets;
    }

    printf("\n--- Proportional Share Scheduling ---\n");
```

```

printf("Enter the Time Period for scheduling: ");
int m;
scanf("%d", &m);

printf("\nExecution Timeline:\n");
printf("-----\n");

// --- Scheduling Simulation ---
for (int i = 0; i < m; i++) {
    int winning_ticket = rand() % total_tickets + 1;
    int accumulated_tickets = 0;
    int winner_index = 0;

    // Find which process holds the winning ticket range
    for (int j = 0; j < n; j++) {
        accumulated_tickets += p[j].tickets;
        if (winning_ticket <= accumulated_tickets) {
            winner_index = j;
            break;
        }
    }
    printf("Time Slot %d -> Ticket Picked: %2d | Winner: %s\n", i + 1, winning_ticket,
p[winner_index].name);
}
printf("-----\n");

// --- Share Probability Breakdown ---
printf("\nExpected Processor Time Share allocation:\n");
for (int i = 0; i < n; i++) {
    printf("Process %s gets %0.2f%% of Processor Time.\n", p[i].name, ((p[i].tickets / total_T
* 100));
}

return 0;
}

```

OUTPUT:

C:\Users\Nitya\Downloads\rr X + v

1BM24CS361, Nitya R Swadi

Enter the number of Processes: 3

Process 1:

Tickets: 10

Process 2:

Tickets: 20

Process 3:

Tickets: 30

--- Proportional Share Scheduling ---

Enter the Time Period for scheduling: 5

Tickets picked: 3, Winner: P1

Tickets picked: 20, Winner: P2

Tickets picked: 54, Winner: P3

Tickets picked: 14, Winner: P2

Tickets picked: 39, Winner: P3

The Process: P1 gets 16.67% of Processor Time.

The Process: P2 gets 33.33% of Processor Time.

The Process: P3 gets 50.00% of Processor Time.

Process returned 0 (0x0) execution time : 34.957 s

Press any key to continue.

Lab Program 5

Producer Consumer problem:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

sem_t empty;      // Semaphore to
count empty slots
sem_t full;        // Semaphore to count
filled slots
pthread_mutex_t mutex; // Mutex lock
for mutual exclusion on buffer
manipulation

void* producer(void* arg) {
    int item, i = 0;
    while (1) {
        item = i++;          // Produce an
item (just a simple incrementing counter)

        sem_wait(&empty);    //
Decrement empty slots (wait if buffer is
```

full)

```
pthread_mutex_lock(&mutex); //
```

Enter critical section

```
buffer[in] = item;
```

```
printf("Produced: %d at
```

```
buffer[%d]\n", item, in);
```

```
in = (in + 1) % BUFFER_SIZE;
```

```
pthread_mutex_unlock(&mutex); //
```

Leave critical section

```
sem_post(&full);          //
```

Increment filled slots (signal consumer)

```
sleep(1); // Simulate time taken to
```

produce an item

```
}
```

```
}
```

```
void* consumer(void* arg) {
```

```
int item;
```

```
while (1) {
```

```
sem_wait(&full);          //
```

Decrement filled slots (wait if buffer is empty)

```
pthread_mutex_lock(&mutex); //
```

Enter critical section

```
item = buffer[out];
```

```
printf("Consumed: %d from
```

```

buffer[%d]\n", item, out);
    out = (out + 1) % BUFFER_SIZE;

    pthread_mutex_unlock(&mutex); //
Leave critical section
    sem_post(&empty);           //
Increment empty slots (signal producer)

    sleep(2); // Simulate time taken to
consume an item (slower than producer)
}
}

int main() {
    // Header preservation
    printf("Nitya R Swadi,
1BM24CS361\n");

    printf("-----
----\n");
    printf("Initializing Bounded-Buffer
System Simulation\n");

    printf("-----
----\n");

    pthread_t prod, cons;

    // Initialize semaphores and mutex
    sem_init(&empty, 0, BUFFER_SIZE);

```

```

// Initial empty slots = size of buffer
    sem_init(&full, 0, 0);          // Initial
filled slots = 0
    pthread_mutex_init(&mutex, NULL);

    // Create the concurrent threads
    pthread_create(&prod, NULL,
producer, NULL);
    pthread_create(&cons, NULL,
consumer, NULL);

    // Wait for the threads to finish
execution (infinite loop in this version)
    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    // Cleanup resources
    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    return 0;
}

```

OUTPUT:

C:\Users\Nitya\Downloads\rm x

+ v

```
Nitya R Swadi, 1BM24CS361
Produced: 0 at buffer[0]
Consumed: 0 from buffer[0]
Produced: 1 at buffer[1]
Consumed: 1 from buffer[1]
Produced: 2 at buffer[2]
Produced: 3 at buffer[3]
Consumed: 2 from buffer[2]
Produced: 4 at buffer[4]
Produced: 5 at buffer[0]
Consumed: 3 from buffer[3]
Produced: 6 at buffer[1]
Produced: 7 at buffer[2]
Consumed: 4 from buffer[4]
Produced: 8 at buffer[3]
Produced: 9 at buffer[4]
Consumed: 5 from buffer[0]
Produced: 10 at buffer[0]
Consumed: 6 from buffer[1]
Produced: 11 at buffer[1]
Consumed: 7 from buffer[2]
Produced: 12 at buffer[2]
|
```


Lab Program 6

Dining Philosophers problem:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define N 5 // Number of philosophers

pthread_mutex_t forks[N]; // One mutex per fork
pthread_t philosophers[N];

void* philosopher(void* num) {
    int id = *(int*)num;
    int left = id;          // Left fork index
    int right = (id + 1) % N; // Right fork index

    while (1) {
        printf("Philosopher %d is thinking.\n", id);
        sleep(1); // Thinking state simulation

        // --- Resource Hierarchy Method to Prevent Deadlock ---
        // Even IDs pick up left fork then right fork.
        // Odd IDs pick up right fork then left fork.
        if (id % 2 == 0) {
            // Pick up left fork
            pthread_mutex_lock(&forks[left]);
            printf("Philosopher %d picked up left fork %d.\n", id, left);

            // Pick up right fork
            pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked up right fork %d.\n", id, right);
        }
        else {
            // Pick up right fork
            pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked up right fork %d.\n", id, right);
```

```

        // Pick up left fork
        pthread_mutex_lock(&forks[left]);
        printf("Philosopher %d picked up left fork %d.\n", id, left);
    }

    // Eating State
    printf("Philosopher %d is eating.\n", id);
    sleep(2);

    // Put down both forks
    pthread_mutex_unlock(&forks[left]);
    pthread_mutex_unlock(&forks[right]);
    printf("Philosopher %d put down forks %d and %d.\n", id, left, right);
}
return NULL;
}

int main() {
    int i;
    int ids[N];

    printf("Dining Philosophers Synchronization Simulation\n");
    printf("-----\n");

    // Initialize forks (mutexes) and IDs
    for (i = 0; i < N; i++) {
        pthread_mutex_init(&forks[i], NULL);
        ids[i] = i;
    }

    // Create philosopher threads
    for (i = 0; i < N; i++) {
        pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
    }

    // Join threads (Keeps the main engine running indefinitely)
    for (i = 0; i < N; i++) {
        pthread_join(philosophers[i], NULL);
    }
}

```

```
// Destroy mutexes (Good cleanup practice, though unreachable here due to infinite loop)
for (i = 0; i < N; i++) {
    pthread_mutex_destroy(&forks[i]);
}

return 0;
}
```

OUTPUT:

```
Nitya R Swadi 1BM24CS361

Philosopher 0 is thinking.
Philosopher 2 is thinking.
Philosopher 3 is thinking.
Philosopher 1 is thinking.
Philosopher 4 is thinking.
Philosopher 1 picked up right fork 2.
Philosopher 1 picked up left fork 1.
Philosopher 1 is eating.
Philosopher 4 picked up left fork 4.
Philosopher 0 picked up left fork 0.
Philosopher 0 picked up right fork 1.
Philosopher 0 is eating.
Philosopher 1 put down forks 1 and 2.
Philosopher 4 picked up right fork 0.
Philosopher 4 is eating.
Philosopher 1 is thinking.
Philosopher 2 picked up left fork 2.
Philosopher 2 picked up right fork 3.
Philosopher 2 is eating.
Philosopher 0 put down forks 0 and 1.
Philosopher 0 is thinking.
Philosopher 2 put down forks 2 and 3.
Philosopher 2 is thinking.
Philosopher 3 picked up right fork 4.
Philosopher 3 picked up left fork 3.
Philosopher 3 is eating.
Philosopher 1 picked up right fork 2.
Philosopher 1 picked up left fork 1.
```

Lab Program 7

Bankers Algorithm

Safety Algorithm:

```
#include <stdio.h>
#include <stdbool.h>

int main() {
    int n, m;

    printf("Banker's Algorithm: Safety
Sequence Finder\n");

    printf("-----
----\n");

    // 1. Input system configurations
    printf("Enter number of processes: ");
    if (scanf("%d", &n) != 1 || n <= 0)
return 1;

    printf("Enter number of resources: ");
    if (scanf("%d", &m) != 1 || m <= 0)
return 1;

    int alloc[n][m];
    int max[n][m];
    int avail[m];
    int need[n][m];
```

```

// 2. Input resource tracking matrices
printf("\nEnter Allocation Matrix (%d
x %d):\n", n, m);
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        scanf("%d", &alloc[i][j]);
    }
}

```

```

printf("\nEnter Maximum Demand
Matrix (%d x %d):\n", n, m);
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        scanf("%d", &max[i][j]);
        // Calculate Need matrix: Need =
Max - Allocation
        need[i][j] = max[i][j] - alloc[i][j];
    }
}

```

```

printf("\nEnter Available Resources
vector (%d elements):\n", m);
for (int j = 0; j < m; j++) {
    scanf("%d", &avail[j]);
}

```

```

// 3. Initialize Safety Algorithm
tracking structures
int work[m];

```

```
for (int j = 0; j < m; j++) {  
    work[j] = avail[j];  
}
```

```
bool finish[n];  
for (int i = 0; i < n; i++) {  
    finish[i] = false;  
}
```

```
int safe_sequence[n];  
int count = 0;
```

// 4. Safety Algorithm Core Engine

Loop

// Maximum possible outer cycles is
bounded by total process count 'n'

```
for (int k = 0; k < n; k++) {  
    for (int i = 0; i < n; i++) {  
        if (!finish[i]) {  
            bool can_allocate = true;
```

// Evaluate resource

availability: Check if Need <= Work for
all instances

```
for (int j = 0; j < m; j++) {  
    if (need[i][j] > work[j]) {  
        can_allocate = false;  
        break;  
    }  
}
```

```

        // If current demands can be
met safely
        if (can_allocate) {
            // Reclaim previously held
simulated resources
            for (int j = 0; j < m; j++) {
                work[j] += alloc[i][j];
            }
            finish[i] = true;
            safe_sequence[count++] = i;
        }
    }
}
}

```

// 5. Output Verification Evaluation

```

printf("\n=====
=====\\n");

printf("      SAFETY
ANALYSIS      \\n");

printf("=====
=====\\n");

if (count == n) {
    printf("STATUS: System is in a
SAFE state.\\n\\n");
    printf("Safe Sequence: ");
    for (int i = 0; i < n; i++) {

```

```

        printf("P%d", safe_sequence[i]);
        if (i < n - 1) {
            printf(" -> ");
        }
    }
    printf("\n");
} else {
    printf("STATUS: System is
UNSAFE! (Deadlock imminent/risk
detected).\n");
}
printf("=====
=====\\n");

return 0;
}

```

OUTPUT:

```

C:\Users\Nitya\Downloads\rrr
Nitya R Swadi 1BM24CS361
Enter number of processes: 5
Enter number of resources: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Maximum Demand Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources:
3 3 2

System is in a safe state.
Safe sequence is: P1 -> P3 -> P4 -> P0 -> P2

Process returned 0 (0x0)   execution time : 110.355 s
Press any key to continue.

```


Deadlock Detection:

```
#include <stdio.h>
#include <stdbool.h>

int main() {
    int n, m;

    printf("Deadlock Detection Algorithm\n\n");

    printf("-----\n");

    // 1. Input system matrix dimensions
    printf("Enter the number of processes:");
    );
    if (scanf("%d", &n) != 1 || n <= 0)
return 1;

    printf("Enter the number of resources:");
    );
    if (scanf("%d", &m) != 1 || m <= 0)
return 1;

    int alloc[n][m];
    int request[n][m];
    int avail[m];

    // 2. Input Allocation Matrix
```

```

printf("\nEnter the Allocation Matrix
(%d x %d):\n", n, m);
for (int i = 0; i < n; i++) {
    printf("Process P%d: ", i);
    for (int j = 0; j < m; j++) {
        scanf("%d", &alloc[i][j]);
    }
}

```

```

// 3. Input Request Matrix
printf("\nEnter the Request Matrix
(%d x %d):\n", n, m);
for (int i = 0; i < n; i++) {
    printf("Process P%d: ", i);
    for (int j = 0; j < m; j++) {
        scanf("%d", &request[i][j]);
    }
}

```

```

// 4. Input Available Resources vector
printf("\nEnter the Available
Resources vector (%d elements): ", m);
for (int j = 0; j < m; j++) {
    scanf("%d", &avail[j]);
}

```

```

// 5. Initialize structures for Detection
Algorithm
int work[m];
for (int j = 0; j < m; j++) {

```

```
    work[j] = avail[j];  
}
```

```
bool finish[n];  
for (int i = 0; i < n; i++) {  
    // Rule 1: If allocation is zero across  
all resources, finish[i] = true  
    bool holds_resources = false;  
    for (int j = 0; j < m; j++) {  
        if (alloc[i][j] > 0) {  
            holds_resources = true;  
            break;  
        }  
    }  
  
    if (!holds_resources) {  
        finish[i] = true;  
    } else {  
        finish[i] = false;  
    }  
}
```

```
int safe_sequence[n];  
int count = 0;
```

```
// 6. Core Detection Engine Loop
```

```
for (int k = 0; k < n; k++) {  
    for (int i = 0; i < n; i++) {  
        if (!finish[i]) {  
            bool can_execute = true;
```

```

// Evaluate resource request
availability: Check if Request <= Work
for (int j = 0; j < m; j++) {
    if (request[i][j] > work[j]) {
        can_execute = false;
        break;
    }
}

```

// If requests can be fulfilled,
simulate completion and reclaim its
assets

```

if (can_execute) {
    for (int j = 0; j < m; j++) {
        work[j] += alloc[i][j];
    }
    finish[i] = true;
    safe_sequence[count++] = i;
}
}
}
}

```

// 7. Evaluate Analysis State Results

```

bool deadlocked = false;
for (int i = 0; i < n; i++) {
    if (!finish[i]) {
        deadlocked = true;
        break;
    }
}

```

```
    }  
}
```

```
printf("\n=====
```

```
=====\\n");
```

```
    printf("        DIAGNOSTIC
```

```
REPORT        \\n");
```

```
printf("=====
```

```
=====\\n");
```

```
    if (!deadlocked) {  
        printf("STATUS: System is in a
```

```
SAFE state.\\n");  
        printf("Safe Execution Sequence:");  
        for (int i = 0; i < count; i++) {  
            printf(" -> P%d",  
safe_sequence[i]);  
        }  
        printf("\\n");  
    } else {  
        printf("STATUS: System is in a
```

```
DEADLOCKED state.\\n");  
        printf("Deadlocked Processes:");  
        for (int i = 0; i < n; i++) {  
            if (!finish[i]) {  
                printf(" P%d", i);  
            }  
        }  
        printf("\\n");
```

```

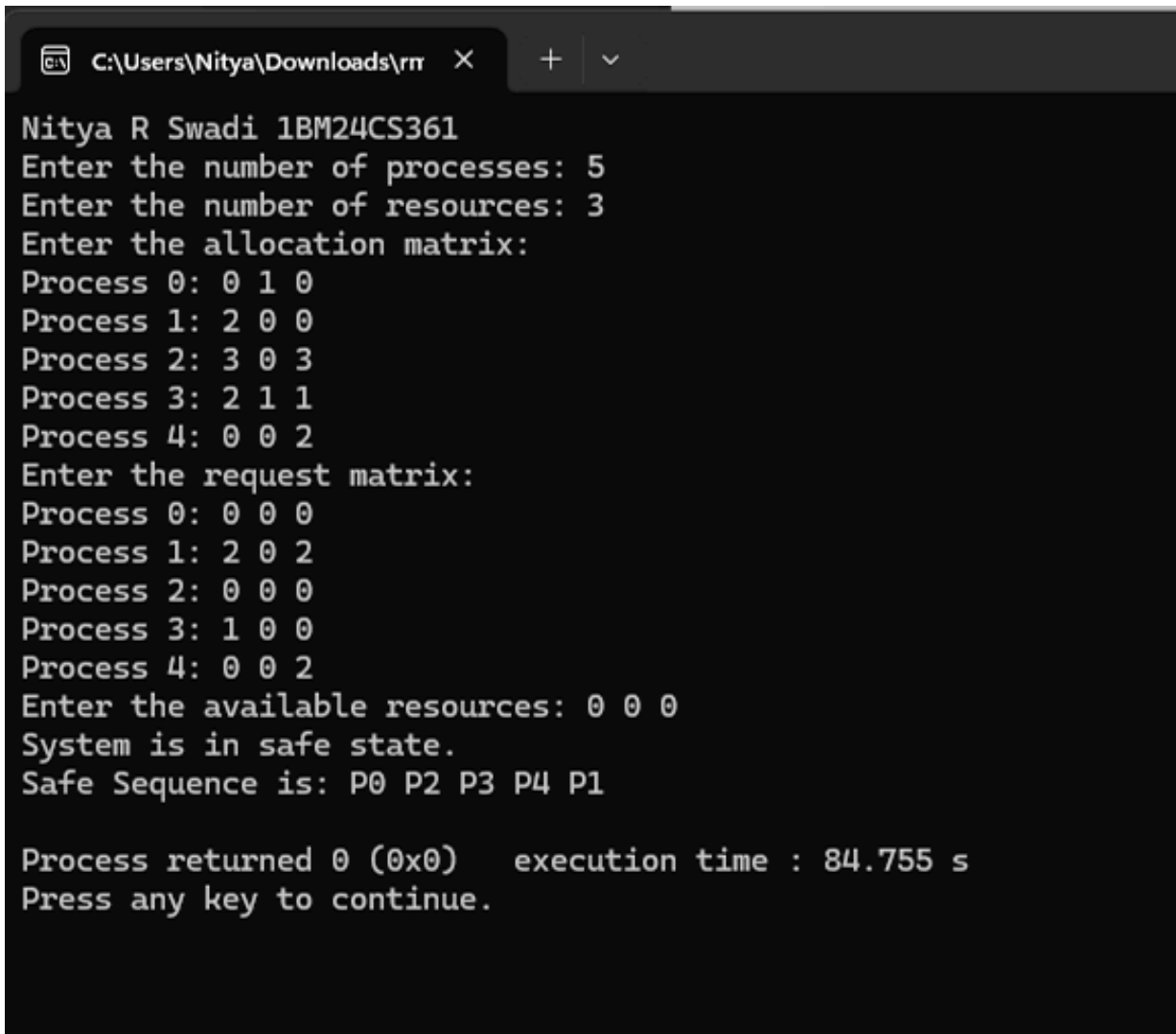
    }

printf("=====
=====\\n");

    return 0;
}

```

OUTPUT:



```

C:\Users\Nitya\Downloads\rrr X + v
Nitya R Swadi 1BM24CS361
Enter the number of processes: 5
Enter the number of resources: 3
Enter the allocation matrix:
Process 0: 0 1 0
Process 1: 2 0 0
Process 2: 3 0 3
Process 3: 2 1 1
Process 4: 0 0 2
Enter the request matrix:
Process 0: 0 0 0
Process 1: 2 0 2
Process 2: 0 0 0
Process 3: 1 0 0
Process 4: 0 0 2
Enter the available resources: 0 0 0
System is in safe state.
Safe Sequence is: P0 P2 P3 P4 P1

Process returned 0 (0x0)    execution time : 84.755 s
Press any key to continue.

```

Lab Program 8

Fitting of memory:

```
#include <stdio.h>

int main() {
    int num_blocks, num_processes;

    // Header preservation
    printf("1BM24CS361\n");

    printf("-----\n");

    printf("Enter number of memory
blocks: ");
    if (scanf("%d", &num_blocks) != 1 ||
num_blocks <= 0) return 1;

    int blocks[num_blocks];
    printf("Enter sizes of %d memory
blocks:\n", num_blocks);
    for (int i = 0; i < num_blocks; i++) {
        scanf("%d", &blocks[i]);
    }

    printf("Enter number of processes: ");
    if (scanf("%d", &num_processes) != 1
|| num_processes <= 0) return 1;
```

```

    int processes[num_processes];
    printf("Enter sizes of %d
processes:\n", num_processes);
    for (int i = 0; i < num_processes; i++)
    {
        scanf("%d", &processes[i]);
    }

```

```

    int allocation[num_processes];
    int used[num_blocks]; // Array
tracking which blocks are already
assigned

```

```

    //
=====

=====

    // STRATEGY 1: FIRST FIT
    //
=====

=====

    for (int i = 0; i < num_blocks; i++)
used[i] = 0;
    for (int i = 0; i < num_processes; i++)
allocation[i] = -1;

```

```

printf("\n=====
=====\\n");
    printf("        FIRST FIT
\\n");

```



```

printf("=====
=====\\n");

printf("Process No.\\tProcess
Size\\tBlock No.\\n");

printf("-----\\
n");

for (int i = 0; i < num_processes; i++)
{
    for (int j = 0; j < num_blocks; j++)
    {
        // Select the very first empty
        block that can accommodate the process
        if (!used[j] && blocks[j] >=
processes[i]) {
            allocation[i] = j + 1;
            used[j] = 1;
            break;
        }
    }
    if (allocation[i] != -1)
        printf("P%d\\t\\t%d\\t\\tBlock
%d\\n", i + 1, processes[i], allocation[i]);
    else
        printf("P%d\\t\\t%d\\t\\tNot
Allocated\\n", i + 1, processes[i]);
}

```

```

//
=====

=====

// STRATEGY 2: BEST FIT
//
=====

=====

    for (int i = 0; i < num_blocks; i++)
used[i] = 0;
    for (int i = 0; i < num_processes; i++)
allocation[i] = -1;


printf("\n=====
=====\\n");
    printf("        BEST FIT
\\n");


printf("=====
=====\\n");
    printf("Process No.\\tProcess
Size\\tBlock No.\\n");


printf("-----\\
n");


    for (int i = 0; i < num_processes; i++)
{
    int best = -1;
    for (int j = 0; j < num_blocks; j++)

```

```

{
    if (!used[j] && blocks[j] >=
processes[i]) {
        // Select the block that
minimizes leftover unallocated space
        if (best == -1 || blocks[j] <
blocks[best])
            best = j;
    }
}
if (best != -1) {
    allocation[i] = best + 1;
    used[best] = 1;
    printf("P%d\t\t%d\t\tBlock
%d\n", i + 1, processes[i], allocation[i]);
} else {
    printf("P%d\t\t%d\t\tNot
Allocated\n", i + 1, processes[i]);
}
}

//
=====

=====

// STRATEGY 3: WORST FIT
//
=====

=====

for (int i = 0; i < num_blocks; i++)
used[i] = 0;

```

```

    for (int i = 0; i < num_processes; i++)
allocation[i] = -1;

```

```

printf("\n=====
=====\\n");

```

```

    printf("        WORST FIT
\\n");

```

```

printf("=====
=====\\n");

```

```

    printf("Process No.\\tProcess
Size\\tBlock No.\\n");

```

```

printf("-----\\
n");

```

```

    for (int i = 0; i < num_processes; i++)
{
    int worst = -1;
    for (int j = 0; j < num_blocks; j++)
{
        if (!used[j] && blocks[j] >=
processes[i]) {
            // Select the maximum size
            block to maximize leftover space
            if (worst == -1 || blocks[j] >
blocks[worst])
                worst = j;
        }
    }
}

```

```

        if (worst != -1) {
            allocation[i] = worst + 1;
            used[worst] = 1;
            printf("P%d\t\t%d\t\tBlock\n", i + 1, processes[i], allocation[i]);
        } else {
            printf("P%d\t\t%d\t\tNot\n", i + 1, processes[i],
                Allocated\n", i + 1, processes[i]);
        }
    }

    printf("-----\n");
    return 0;
}

```

OUTPUT:

```

C:\Users\Admin\Desktop\wor X
18M24CS361

Enter number of memory blocks: 5
Enter sizes of 5 memory blocks:
400 700 200 300 600
Enter number of processes: 4
Enter sizes of 4 processes:
212 512 312 526

--- First Fit ---
Process No.    Process Size    Block No.
1              212          1
2              512          2
3              312          5
4              526        Not Allocated

--- Best Fit ---
Process No.    Process Size    Block No.
1              212          4
2              512          5
3              312          1
4              526          2

--- Worst Fit ---
Process No.    Process Size    Block No.
1              212          2
2              512          5
3              312          1
4              526        Not Allocated

Process returned 0 (0x0)   execution time : 34.948 s
Press any key to continue.

```

Lab Program 9

Page replacement:

```
#include <stdio.h>

#define MAX_FRAMES 10
#define MAX_PAGES 50

// =====
// Utility: Print current frame state
// =====
void printFrames(int frames[], int num_frames) {
    printf("[");
    for (int i = 0; i < num_frames; i++) {
        if (frames[i] == -1)
            printf(" -");
        else
            printf(" %2d", frames[i]);
    }
    printf(" ]");
}

// =====
// 1. FIFO Page Replacement
// =====
void fifo(int pages[], int n, int num_frames) {
    int frames[MAX_FRAMES];
    int page_faults = 0;
    int next = 0; // Pointer to the oldest frame

    for (int i = 0; i < num_frames; i++) frames[i] = -1;

    printf("\n=====");
    printf("\n    FIFO Page Replacement    ");
    printf("\n=====\\n");

    for (int i = 0; i < n; i++) {
        int page = pages[i];
        int found = 0;
```

```

// Check if page is already in frames
for (int j = 0; j < num_frames; j++) {
    if (frames[j] == page) {
        found = 1;
        break;
    }
}

if (!found) {
    frames[next] = page;
    next = (next + 1) % num_frames;
    page_faults++;
}

printf("Page %2d -> ", page);
printFrames(frames, num_frames);
printf("%s\n", found ? "" : "[Fault *]");
}
printf("-----\n");
printf("Total Page Faults (FIFO): %d\n", page_faults);
printf("-----\n");
}

// =====
// 2. Optimal Page Replacement
// =====

void optimal(int pages[], int n, int num_frames) {
    int frames[MAX_FRAMES];
    int page_faults = 0;

    for (int i = 0; i < num_frames; i++) frames[i] = -1;

    printf("\n=====");
    printf("\n    Optimal Page Replacement    ");
    printf("\n=====\\n");

    for (int i = 0; i < n; i++) {
        int page = pages[i];
        int found = 0;

```

```

// Check if page is already in frames
for (int j = 0; j < num_frames; j++) {
    if (frames[j] == page) {
        found = 1;
        break;
    }
}

if (!found) {
    // Find empty slot first
    int empty = -1;
    for (int j = 0; j < num_frames; j++) {
        if (frames[j] == -1) {
            empty = j;
            break;
        }
    }

    if (empty != -1) {
        frames[empty] = page;
    } else {
        // Find the page used farthest in the future
        int replace = -1, farthest = -1;
        for (int j = 0; j < num_frames; j++) {
            int next_use = n; // Assume never used again
            for (int k = i + 1; k < n; k++) {
                if (pages[k] == frames[j]) {
                    next_use = k;
                    break;
                }
            }
            if (next_use > farthest) {
                farthest = next_use;
                replace = j;
            }
        }
        frames[replace] = page;
    }
    page_faults++;
}

```



```

    }

    printf("Page %2d -> ", page);
    printFrames(frames, num_frames);
    printf("%s\n", found ? "" : " [Fault *]");
}
printf("-----\n");
printf("Total Page Faults (Optimal): %d\n", page_faults);
printf("-----\n");
}

// =====
// 3. LRU Page Replacement
// =====

void lru(int pages[], int n, int num_frames) {
    int frames[MAX_FRAMES];
    int last_used[MAX_FRAMES]; // Tracks the index step when each frame was last hit
    int page_faults = 0;

    for (int i = 0; i < num_frames; i++) {
        frames[i] = -1;
        last_used[i] = -1;
    }

    printf("\n=====");
    printf("\n    LRU Page Replacement    ");
    printf("\n=====\\n");

    for (int i = 0; i < n; i++) {
        int page = pages[i];
        int found = 0, found_idx = -1;

        // Check if page is already in frames
        for (int j = 0; j < num_frames; j++) {
            if (frames[j] == page) {
                found = 1;
                found_idx = j;
                break;
            }
        }
    }
}

```

```

if (found) {
    last_used[found_idx] = i; // Update last used reference time marker
} else {
    // Find empty slot first
    int empty = -1;
    for (int j = 0; j < num_frames; j++) {
        if (frames[j] == -1) {
            empty = j;
            break;
        }
    }

    if (empty != -1) {
        frames[empty] = page;
        last_used[empty] = i;
    } else {
        // Find the least recently used frame
        int lru_idx = 0;
        for (int j = 1; j < num_frames; j++) {
            if (last_used[j] < last_used[lru_idx])
                lru_idx = j;
        }
        frames[lru_idx] = page;
        last_used[lru_idx] = i;
    }
    page_faults++;
}

printf("Page %2d -> ", page);
printFrames(frames, num_frames);
printf("%s\n", found ? "" : " [Fault *]");
}
printf("-----\n");
printf("Total Page Faults (LRU): %d\n", page_faults);
printf("-----\n");
}

// =====
// Main Engine

```

```
// =====
int main() {
    int n, num_frames;
    int pages[MAX_PAGES];

    printf("Page Replacement Execution Simulator\n");
    printf("-----\n");

    printf("Enter number of pages: ");
    if (scanf("%d", &n) != 1 || n <= 0) return 1;

    printf("Enter page reference string (space-separated):\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &pages[i]);
    }

    printf("Enter number of frames: ");
    if (scanf("%d", &num_frames) != 1 || num_frames <= 0) return 1;

    // Run simulation for all three strategies
    fifo(pages, n, num_frames);
    optimal(pages, n, num_frames);
    lru(pages, n, num_frames);

    return 0;
}
```

OUTPUT:

Nitya R Swadi 1BM24CS361

Page Replacement Execution Simulator

Enter number of pages: 7

Enter page reference string (space-separated):

1 3 0 3 5 6 3

Enter number of frames: 3

FIFO Page Replacement

```
Page 1 -> [ 1 - - ] [Fault *]
Page 3 -> [ 1 3 - ] [Fault *]
Page 0 -> [ 1 3 0 ] [Fault *]
Page 3 -> [ 1 3 0 ]
Page 5 -> [ 5 3 0 ] [Fault *]
Page 6 -> [ 5 6 0 ] [Fault *]
Page 3 -> [ 5 6 3 ] [Fault *]
```

Total Page Faults (FIFO): 6

Optimal Page Replacement

```
Page 1 -> [ 1 - - ] [Fault *]
Page 3 -> [ 1 3 - ] [Fault *]
Page 0 -> [ 1 3 0 ] [Fault *]
Page 3 -> [ 1 3 0 ]
Page 5 -> [ 5 3 0 ] [Fault *]
Page 6 -> [ 6 3 0 ] [Fault *]
Page 3 -> [ 6 3 0 ]
```

Total Page Faults (Optimal): 5

=====

LRU Page Replacement

=====

Page 1 -> [1 - -] [Fault *]

Page 3 -> [1 3 -] [Fault *]

Page 0 -> [1 3 0] [Fault *]

Page 3 -> [1 3 0]

Page 5 -> [5 3 0] [Fault *]

Page 6 -> [5 3 6] [Fault *]

Page 3 -> [5 3 6]

Total Page Faults (LRU): 5

Process returned 0 (0x0) execution time : 53.447 s

Press any key to continue.

|